Shared-noise blending in NonstationaryVariogram

Varying azimuth in gaussianfft: model, implementation and limitations

2026-10-28

Contents

1 Purpose 1

2 Model 2
- 2.1 Help azimuths and blending weights . . . 2
- 2.2 Independent noise versus shared noise . . . 2
- 2.3 Normalisation . . . 3

3 Kriging: `corr_matrix` 3
- 3.1 Rotation trick . . . 3
- 3.2 Lookup . . . 3
- 3.3 Cross-covariance tables: `_cross_tables(ndim)` . . . 4

4 Simulation: `simulate(grid)` 4

5 Kriging and conditional simulation 4

6 Usage 5
- 6.1 Parameters . . . 5

7 Accuracy 6

8 Cost 6

9 Limitations and known issues 6

10 Relation to other approaches 7

11 Tests 7

1 Purpose

`NonstationaryVariogram` (`gaussianfft/_kriging/nonstationary_variogram.py`) is a covariance model whose anisotropy azimuth varies with location. It can be used in:

- unconditional simulation;
- simple and ordinary kriging;
- conditional simulation.

The FFT simulator in gaussianfft is stationary, so the model is built by blending stationary *component* fields with fixed azimuths. This document covers the `shared_noise=True` mode. In this mode all components are driven by the same white noise, so correlation carries across azimuth bins. In the default mode the components are independent.

The document covers the model, the implementation of kriging (`corr_matrix`) and of simulation (`simulate`), and the accuracy, cost and limitations measured so far.

2 Model

2.1 Help azimuths and blending weights

The azimuth circle is divided into N = `n_bins` *help azimuths* $\theta_m = m \cdot 360°/N$, $m = 0, \dots, N-1$. Component m is the stationary variogram C_m with azimuth θ_m. All other parameters (type, `main_range`, `perp_range`, `depth_range`, dip, power) are shared by all components.

The user-supplied function `azimuth(locations)` gives an azimuth $\theta(s)$ at each location s, in degrees counter-clockwise from the x axis (the `gaussianfft.variogram` convention). If $\theta(s)$ lies in $[\theta_a, \theta_b)$, the location gets two linear weights:

$$\tilde{w}_a = \frac{\theta_b - \theta}{\theta_b - \theta_a}, \qquad \tilde{w}_b = \frac{\theta - \theta_a}{\theta_b - \theta_a}.$$

All other components get weight 0. The field is

$$Z(s) = w_a(s)\, Z_a(s) + w_b(s)\, Z_b(s),$$

where Z_m is a unit-variance stationary field with covariance C_m, and w is $\tilde{w}$ normalised so that Var $Z(s) = 1$ (see Normalisation).

`NonstationaryVariogram.weights(locations)` returns, for each location, the two component indices (a, b) and the two normalised weights.

2.2 Independent noise versus shared noise

With **independent noise** (the default), Z_m and Z_n are independent for $m \neq n$. The covariance is then

$$\text{Cov}\left(Z(s), Z(s')\right) = \sum_m w_m(s)\, w_m(s')\, C_m(s - s').$$

Two locations correlate only if they share a component. Locations whose azimuths are more than one bin apart are therefore uncorrelated, even when they are close to each other. As a result, a fine binning (large N) breaks up the field wherever the azimuth changes.

With **shared noise**, every component is a filtered version of the *same* white noise W:

$$Z_m = \mathcal{F}^{-1}\left[\sqrt{S_m}\, \mathcal{F}W\right],$$

where S_m is the spectral density of C_m. The cross-covariance between two components is then

$$C_{mn}(h) = \text{Cov}\left(Z_m(s+h), Z_n(s)\right) = \mathcal{F}^{-1}\left[\sqrt{S_m S_n}\right](h),$$

and the covariance of the blended field is

$$\text{Cov}\left(Z(s), Z(s')\right) = \sum_{m,n} w_m(s)\, w_n(s')\, C_{mn}(s - s').$$

Neighbouring components are strongly correlated: $C_{m,m+1}(0) \approx 0.94$ for $N = 36$. Correlation therefore carries smoothly across bin boundaries, and a fine binning does not break up the field.

The model is a valid (positive semi-definite) covariance by construction, because it is the covariance of an actual linear functional of W.

2.3 Normalisation

With shared noise, the variance at a location is

$$\text{Var } Z(s) = \tilde{w}_a^2 + \tilde{w}_b^2 + 2\,\tilde{w}_a \tilde{w}_b\,\rho, \qquad \rho = C_{a,a+1}(0).$$

All components are rotations of each other, so ρ is the same for every adjacent pair. The weights are divided by the square root of this variance. With independent noise $\rho = 0$, which gives $\sqrt{\tilde{w}_a^2 + \tilde{w}_b^2}$.

In code, ρ = `T_1(0)` is read from the cross-covariance tables described below. For `n_bins = 1`, $T_0(0) = 1$ is used instead.

3 Kriging: `corr_matrix`

`corr_matrix(points, locations)` returns the correlation matrix that the kriging toolkit uses. With `shared_noise=True` it dispatches to `_shared_corr_matrix`.

3.1 Rotation trick

The function C_{mn} has no closed form, so it is tabulated. Storing all N^2 pairs is not needed: azimuth rotates about the z axis, so

$$S_m(\omega) = S_0(R(-\theta_m)\,\omega),$$

where $R(\alpha)$ rotates the horizontal components by α and leaves the vertical component unchanged. If n is k bins ahead of m, then

$$C_{mn}(h) = T_k(R(-\theta_m)\,h), \qquad T_k = \mathcal{F}^{-1}\left[\sqrt{S_0\,S_k}\right].$$

Only the tables T_k are needed, and only for $k = 0, \ldots, \lfloor N/2 \rfloor$: any pair is at most $N/2$ bins apart, counting in the shorter direction.

For an unordered pair $\{m, n\}$, the implementation chooses a *base* b:

- b is the member from which the other is $k \le N/2$ bins ahead;
- when $k = N/2$ exactly, b is the smaller index.

Since $\sqrt{S_0 S_k}$ is real and even, T_k is even and $T_k(-h) = T_k(h)$. The base therefore depends only on the pair, and the order of points and locations does not matter. This makes `corr_matrix(P, L)` exactly the transpose of `corr_matrix(L, P)`, which kriging requires.

The vertical component of the lag is not rotated, so the trick also holds with non-zero dip. The dip is defined relative to the main axis of each component, which is how gaussianfft rotates azimuth.

3.2 Lookup

For each location s' and each of its two components m, and for each of the two component slots n of all points s, `_shared_corr_matrix`:

1. finds k and the base b of $\{m, n\}$;
2. rotates the lag $h = s - s'$ by $-\theta_b$;
3. reads $T_k(R(-\theta_b)h)$ from the table by multilinear interpolation (`scipy.ndimage.map_coordinates`, `order=1`), with 0 outside the table;
4. adds $w_m(s')\,w_n(s)\,T_k(\cdot)$ to the matrix.

3.3 Cross-covariance tables: `_cross_tables(ndim)`

There is one table set per dimension, built on first use and cached:

- 2D points use 2D tables of shape `(N/2+1, L, L)` with `L = lattice_size` (default 512).
- 3D points use 3D tables of shape `(N/2+1, Lh, Lh, Lv)` with `(Lh, Lv) = lattice_size_3d` (default `(256, 64)`).

Each table T_k is built in four steps:

1. Evaluate C_k on the lag lattice with `gaussianfft.Variogram.corr_array`.
2. Compute the spectrum with `rfftn` and clip negative values to 0. Values are negative only because of truncation and round-off.
3. Compute T_k = `irfftn(sqrt(S_0 * S_k))`.
4. Normalise all tables by $T_0(0)$ and shift lag 0 to the centre with `fftshift`.

The tables are stored as `float32`.

The lattice is set up as follows. Here a is `main_range`, $a_\perp$ is `perp_range`, a_z is `depth_range` and δ is the dip.

Quantity	Value
Horizontal half-extent	$4 \max(a, a_\perp, a_z \lvert\sin\delta\rvert)$
Horizontal spacing	$\max\left(\min(a, a_\perp)/8,\ 2 \cdot \text{half-extent}/L_h\right)$
Vertical half-extent	$4\sqrt{(a\sin\delta)^2 + (a_z\cos\delta)^2}$
Vertical range	$r_z = 1/\sqrt{(\sin\delta/a)^2 + (\cos\delta/a_z)^2}$
Vertical spacing	$\max\left(r_z/8,\ 2 \cdot \text{half-extent}_z/L_v\right)$

The extent of 4 ranges covers lags where the correlation is effectively 0. The spacing is at least 1/8 of the shortest range. A finer spacing would mostly cost memory, since linear interpolation already limits the accuracy. When the lattice size is too small to reach that spacing over the full extent, the extent wins and the spacing becomes coarser (see Accuracy).

4 Simulation: `simulate(grid)`

`simulate` builds one realisation as $\sum_m w_m(s)\, Z_m(s)$, using only the components that have a non-zero weight somewhere on the grid. With shared noise, three things make every component see the same white noise:

1. **Same seed.** The realisation reads the current seed (`gaussianfft.seed()`) and sets that seed before each component, so every component draws the same noise. Afterwards the seed is set to `seed + 1`. Consecutive realisations, for example inside `SimpleKriging.simulate(..., n_sim=...)`, therefore use seeds $s_0, s_0+1, s_0+2, \ldots$, and each one is reproducible.
2. **Same padded grid.** gaussianfft chooses the FFT padding from the variogram, so rotated components would otherwise get different FFT sizes and different noise. The padding is the maximum of `gaussianfft.simulation_size` over the components used. Each component is then simulated with `gaussianfft.advanced.simulate(..., padx=, pady=, padz=)`.
3. **3D grids always use a 3D FFT.** gaussianfft runs a 2D FFT when `nz <= 1` (`src/gaussfft.cpp`). A 2D FFT of a single layer has different shared-noise cross-covariances from a section of a 3D field: slightly different at dip 0 (up to 0.008) and strongly different with dip (up to 0.5 at dip 40). The 3D tables describe the section of a 3D field. A single-layer 3D grid is therefore simulated with two layers, and layer 0 is returned.

A 2D grid (`grid.shape` of length 2) keeps the 2D FFT and the 2D tables. The 2D tables describe that case.

A single field's covariance is the same in 2D and in 3D, so `Variogram.simulate` and independent-noise `NonstationaryVariogram.simulate` keep gaussianfft's 2D FFT for a single layer. Their results are unchanged from before.

5 Kriging and conditional simulation

`kriging_toolkit.py` (`SimpleKriging`, `OrdinaryKriging`) works only with the abstract `Covariance` interface:

- `corr_matrix(points, locations)` is used for the kriging system and the prediction weights;
- `simulate(grid)` is used for the unconditional fields in conditional simulation.

Conditional simulation adds the kriged residual to an unconditional field. It is correct only if `simulate` reproduces `corr_matrix`. This is why simulation must use shared noise, a common padded grid and the 3D FFT, as described above.

Raw `gaussianfft.Variogram` objects are converted to the `Variogram` wrapper in the API layer (`_kriging/api.py`), before they reach the toolkit.

6 Usage

```python
import numpy as np
import gaussianfft as grf
from gaussianfft._kriging import GridSpec, NonstationaryVariogram, ObservationData, SimpleKriging

def rotating_azimuth(locations, centre=(495.0, 495.0)):
    angle = np.degrees(np.arctan2(locations[:, 1] - centre[1], locations[:, 0] - centre[0]))
    return (angle + 90.0) % 360.0

grid = GridSpec((100, 100, 5), (10.0, 10.0, 5.0))
variogram = NonstationaryVariogram(
    'matern52', main_range=300, perp_range=60, depth_range=20,
    azimuth=rotating_azimuth, dip=10, n_bins=36, shared_noise=True,
)

grf.seed(42)
uncond = variogram.simulate(grid)                       # (100, 100, 5)

obs = ObservationData(obs_locations, obs_values, obs_uncertainties)  # your data
sk = SimpleKriging(mean=0.0)
mean, _ = sk.predict(grid, obs, variogram)
sims = sk.simulate(grid, obs, variogram, n_sim=10)      # seeds 42, 43, ...
```

Azimuth given on a grid can be wrapped with `GriddedAzimuth(values, grid)`. It interpolates unit vectors, so it handles the wrap at 360°.

A full example is in `examples/nonstationary_azimuth_grf.py`. It shows the azimuth, the unconditional simulation, the SK prediction and a conditional simulation in layers 1 and 5.

6.1 Parameters

Parameter	Default	Meaning
`type`		gaussianfft variogram type, e.g. `'matern52'`
`main_range`		Main range
`perp_range`	`main_range`	Perpendicular range
`depth_range`	`main_range`	Vertical range
`azimuth`		Function `(n, ndim) -> (n,)` returning degrees
`dip`	0	Dip in degrees
`power`	1.5	Exponent for general exponential variograms
`n_bins`	100	Number of help azimuths over 360°
`shared_noise`	`False`	Drive all components with the same white noise
`lattice_size`	512	Table nodes per axis for 2D locations
`lattice_size_3d`	`(256, 64)`	Table nodes (horizontal, vertical) for 3D locations

7 Accuracy

The tabulated model was compared with the exact stationary variogram for a constant azimuth on a bin. In that case only T_0 is used and the model should equal C_0, so the error measures the table resolution alone. The setup was `matern52` 300/60/20, 300 random 3D points in 1000 × 1000 × 20 m, and `n_bins = 36`.

Dip	`lattice_size_3d`	Spacing (x, y, z) m	Max abs error
0	(256, 64)	9.4, 9.4, 2.5	0.042
0	(512, 64)	7.5, 7.5, 2.5	0.032
10	(256, 64)	9.4, 9.4, 7.0	0.154
10	(256, 256)	9.4, 9.4, 2.5	0.050
10	(512, 256)	7.5, 7.5, 2.5	0.037
40	(256, 64)	9.4, 9.4, 24	0.53

An azimuth halfway between two bins gives about the same errors (0.049, 0.156 and 0.53 for dip 0, 10 and 40 with the defaults). Bin blending therefore adds little; the lattice resolution dominates.

Conclusions from this setup:

- The main error source is the vertical spacing when the dip is non-zero. The main axis then has a vertical extent of about $4a \sin\delta$, much larger than a_z, while correlation across that axis varies on the scale $r_z \approx a_z$. With the default $L_v = 64$ the spacing becomes about $r_z/3$. The error at dip 10 drops from 0.15 to 0.05 with $L_v = 256$.
- The error floor of about 0.03 comes from linear interpolation at a spacing of 1/8 of the shortest range.
- The tests use a milder setup: 100/30/15, dip 10, `n_bins = 18` and `lattice_size_3d = (128, 64)`. For a constant azimuth on a bin they pass with tolerance 0.05.
- The empirical covariance of 300 to 600 shared simulations on a layered grid matches `corr_matrix` within about 0.035, with variance about 1.

When the dip is non-zero, increase `lattice_size_3d[1]` until $2 \cdot 4\sqrt{(a\sin\delta)^2 + (a_z\cos\delta)^2}/L_v \le r_z/8$. Memory grows linearly with it.

8 Cost

Item	Size / time
3D tables, `n_bins = 36`, (256, 64)	19 × 256 × 256 × 64 × 4 B ≈ 320 MB, built in ≈ 5 s
3D tables, `n_bins = 100`, (256, 64)	≈ 860 MB
2D tables, `n_bins = 100`, 512	51 × 512 × 512 × 4 B ≈ 53 MB
Example (100 × 100 × 5, 36 bins, simulate + SK + 1 cond. sim.)	≈ 8 s

Memory is proportional to `(n_bins/2 + 1) · Lh² · Lv`. Simulation costs one padded FFT per component used on the grid. On a single layer, the extra layer and the vertical padding add to the cost.

9 Limitations and known issues

- **Table resolution with dip.** The default `lattice_size_3d = (256, 64)` is too coarse vertically for a steep dip or a large `main_range/depth_range` ratio. See Accuracy. Raising L_v helps at a linear memory cost.
- **Memory.** 3D tables with many bins take hundreds of MB.
- **Single layer is padded vertically.** A single-layer 3D grid is simulated as a 3D field with vertical padding of about the vertical correlation length divided by `dz`. Use a realistic `depth_range` and `dz`. With the default `depth_range = main_range` and a small `dz`, the FFT becomes very large.

- **Redundant bins.** Azimuth is axial: θ and $\theta + 180°$ give the same spectrum. With bins over 360°, components m and $m + N/2$ are identical fields under shared noise. The effective number of distinct directions is $N/2$, so half of the components and FFTs are redundant. Binning over 180° would halve the cost at equal resolution.
- **1D locations** are not supported with shared noise (`NotImplementedError`).
- **The azimuth should still vary slowly** relative to the range. Blending linearly between two components approximates a locally stationary field at the local azimuth, not an exact non-stationary covariance.

10 Relation to other approaches

The implementation is the *anchor* approach: fixed help azimuths, linear blending between adjacent anchors, and shared noise to correlate the anchors. A *steerable* expansion in angular harmonics represents the rotation more efficiently for a given accuracy, but it is harder to explain and to check. The anchor approach is the more transparent of the two: every component is an ordinary gaussianfft variogram and can be inspected on its own. Binning over 180° (see above) is the simplest improvement within this approach.

11 Tests

The tests are in `tests/test_nonstationary_variogram.py`.

`TestSharedNoise` (2D) covers:

- empirical covariance of simulations versus `corr_matrix`;
- unit variance, symmetry and positive definiteness;
- correlation across bins;
- a constant azimuth on a bin versus the stationary model;
- reproducible and distinct realisations (seed + 1);
- SK and OK conditional simulation honouring the observations;
- `NotImplementedError` for 1D locations.

`TestSharedNoise3D` uses a 24 × 24 × 4 grid with `dz = 5`, dip 10 and `n_bins = 18`. It covers:

- components being rotations about z;
- a constant azimuth versus the stationary model;
- empirical covariance on a layered grid;
- a single layer being a section of the 3D field;
- symmetry and positive definiteness;
- SK and OK conditional simulation with observations in several layers.